

Vision-Based PDF RAG with ColQwen2 and Gemini: Build Guide

Built from the learnbayofficial carousel "Build Vision-Based PDF RAG Using ColPali and ColQwen2" (Instagram, posted 2026-09-15), then checked against the libraries' own docs, PyPI, Hugging Face and Google's Gemini docs on 2026-09-28. Where the carousel and those sources disagree, this document follows the sources and says so.

1. What you are building

A command-line tool that lets you ask questions of a PDF and get answers that cite the page they came from. It never extracts the PDF's text. Each page is rendered as an image and embedded with the vision retriever `vidore/colqwen2-v1.0`, which stores one 128-number vector per image patch. A question is embedded the same way, and every page is scored by `MaxSim` (late interaction). The top pages are then re-rendered and sent as images, along with the question, to Gemini. Gemini is told to answer only from those pages and to cite (p. N).

With it you can ask things like "What was revenue growth between Q2 and Q3?" against a table, "Which year has the highest outlay?" against a chart, or "What does the footnote under figure 3 say?" against a multi-column page. Each answer includes the page numbers you can use to check it.

The carousel showed the indexing half and the retrieval function. Its last slide held back the answering half behind "comment RAG". This document writes that half from the official docs: loading the retrieved page images, the Gemini call, the grounding prompt, the CLI and an end-to-end run.

2. Decision box: is this worth building?

Your situation	Do this	Why (verified numbers)
One PDF, up to 1,000 pages and 50 MB , tens of questions	Just send the whole PDF to Gemini (<code>baseline.py</code> below)	Gemini accepts PDFs "up to 50MB or 1000 pages" and counts "258 tokens" per page. <code>gemini-3.8-flash</code> has a 1,048,576-token input window. On the free tier, input and output are "Free of charge". On the paid tier the cost is about \$0.0002 per page per question (258 x \$0.75 per 1M), so a 300-page PDF costs about \$0.06 per question.
The same PDF, many repeated questions	Whole PDF plus context caching	Implicit caching is on by default for 2.5-and-newer models. Explicit cache reads cost \$0.075 per 1M tokens (10x cheaper than input), and storage costs \$0.50 per 1M tokens per hour. The cache minimum is 4,096 tokens, which is about 16 pages or more.
A PDF over 1,000 pages or 50 MB , or a collection of PDFs to search across, or hundreds of questions against a 500+ page document	Build this project	Whole-PDF sending is impossible past the limit. This pipeline sends only the top 3 pages: about $3 \times 1,548 = \sim 4,650$ input tokens, or about \$0.0035 per question plus output (estimate; see section 5.4). A 500-page whole-PDF call is $\sim 129,000$ tokens, about \$0.10 per question.
Scanned pages, handwriting, odd fonts, charts where the layout carries the meaning	Build this project	This is where page-image retrieval clearly beats text extraction, per the parser-versus-retriever comparison in the research. Even good layout-aware parsers struggle here.

Your situation	Do this	Why (verified numbers)
Born-digital PDFs (exported from Word or LaTeX, not scanned) with regular tables, a large corpus, CPU only, or a need for sentence-level citations or keyword search	Layout-aware text pipeline instead	Docling (MIT, local, free; its GraniteDocling model self-reports table TEDS 0.97), marker (local; weights free under \$5M revenue or funding), unstructured (Apache 2.0 local, or cloud at \$0.015/page after 10,000 free pages), Mistral OCR 4.1 (API, \$4 per 1,000 pages).

For a single ordinary PDF, the whole-PDF baseline will probably answer as well as this pipeline, with none of the setup. Run `baseline.py` on your evaluation questions (section 10) before and after building. The project is worth keeping if it wins on your documents, or if your documents are past the 1,000-page / 50 MB line.

Paid prices above are the `gemini-3.8-flash` rates **through 2026-12-31**. From 2027-01-01 input goes to \$1.50 and output to \$7.50 per 1M tokens, which doubles every figure.

baseline.py (the do-nothing alternative)

This follows Google's Files API sample (document-processing page), with a grounding instruction added:

```
# baseline.py - send the whole PDF to Gemini, no retrieval
# Usage: python baseline.py pdfs/report.pdf "What was revenue growth between Q2 and Q3?"
import sys

from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()
client = genai.Client() # reads GEMINI_API_KEY

pdf_path, question = sys.argv[1], sys.argv[2]
uploaded = client.files.upload(file=pdf_path)
response = client.models.generate_content(
    model="gemini-3.8-flash",
    contents=[uploaded, question],
    config=types.GenerateContentConfig(
        system_instruction="Answer only from this document. Cite the page number for each fact as (p. N).",
    ),
)
print(response.text)
```

3. What the carousel got wrong or left stale

Carousel says	Verified on 2026-09-28	What this guide does
Slide 2: transformers 5.7.0. Slide 7: "Pin transformers==5.3.0; 5.4+ breaks with ImportError: cannot import name '_maybe_shard_state_dict_for_tp'"	The two slides contradict each other. 5.3.0 is colpali-engine's minimum , not a ceiling: 0.3.18 declares transformers>=5.3.0,<6.0.0. The ImportError string is real, but it came from an unrelated, closed issue: huggingface/transformers #45405, a dev build 5.6.0.dev0 paired with a peft that was too old. No colpali-engine or ColQwen2 report of it exists.	Pins transformers==5.17.0 (current), inside colpali-engine's range

Carousel says	Verified on 2026-09-28	What this guide does
colpali-engine 0.3.15	Current is 0.3.18 (2026-08-22). 0.3.18 fixed ColQwen2 checkpoint conversion so LoRA weights are not silently dropped or reinitialised under transformers v5. The maintainers also now mark colpali-engine deprecated and recommend Sentence Transformers v6 MultiVectorEncoder for new work.	Pins 0.3.18. The Sentence Transformers route is covered in Extensions.
torch 2.10	Current torch is 2.14.0, but colpali-engine 0.3.18 requires torch<2.14.0	Pins torch==2.13.0 + torchvision==0.28.0
google-genai 1.75.0	Current is 2.25.0 (2026-09-22). The old google-generativeai package is deprecated.	Pins 2.25.0
"Gemini 2.5 Flash, free tier"	gemini-2.5-flash is now "access limited" to accounts that used it before. Google's models page says to use 3.5 Flash-Lite or 3.8 Flash for new projects.	Uses gemini-3.8-flash. gemini-3.5-flash-lite is the cheaper option.
"First run will download the model (~5GB)"	About 8.9 GB . The adapter is ~90 MB, but it pulls vidore/colqwen2-base, whose float32 shards total 8,836,808,464 bytes.	Plan disk for ~9 GB of model cache
"Colab T4 16GB or any GPU with 8GB+ VRAM"	Neither the model card nor colpali-engine states any VRAM figure. 8 GB is unconfirmed .	Targets a 16 GB T4 and prints peak VRAM so you can measure it
"DPI 300 uses significantly more GPU memory"	The processor caps each page at max_pixels = 602112, which is 768 patches. A 150 DPI letter page (about 2.1M pixels) is already scaled down to that cap before the model sees it. DPI above ~80 changes CPU RAM and render time, not model VRAM.	Keeps 150 DPI, because it matters for the image Gemini reads
fmt="RGB" "handles RGBA"	In pdf2image, fmt is the image file format (ppm, jpeg, png), not a colour mode	Drops fmt and calls .convert("RGB") explicitly
np.save(..., emb.numpy()) on a bfloat16 GPU embedding	NumPy has no bfloat16 type. PyTorch refuses to convert a bfloat16 tensor with .numpy() (TypeError), so the slide code fails on GPU.	Casts to float16 before saving
Windows poppler example Release-25.07.0-0	Latest is Release-26.09.0-0.zip (tag v26.09.0-0, 2026-09-15). The steps are unchanged.	Uses the current name
vidore/colqwen2-v1.0: Apache 2.0, no token	Correct. license: apache-2.0, gated: false	Unchanged

4. Prerequisites

4.1 Hardware

Item	Figure	Status
GPU	NVIDIA with CUDA. A Colab T4 (16 GB) is the reference target.	Carousel claim; T4 is 16 GB
Minimum VRAM	The model card and colpali-engine docs are silent. Rough arithmetic: the base weights are 8.84 GB in float32, so about 4.4 GB in bfloat16, plus activations for each batch.	Arithmetic, not a documented figure. <code>ingest.py</code> prints your real peak.
Model download / disk	~8.9 GB in the Hugging Face cache	Verified from file sizes
Index on disk	~192 KiB per page (768 vectors x 128 x 2 bytes, float16)	Arithmetic from model config
CPU-only	Works in float32. Needs about 8.8 GB RAM for weights alone, so use a machine with 16 GB+. Speed not measured.	Arithmetic; speed unverified
RAM while rendering	A 150 DPI letter page is 1275 x 1650 x 3 bytes = ~6.3 MB. <code>ingest.py</code> renders 16 pages at a time.	Arithmetic
Apple Silicon	This code runs on CPU on a Mac. MPS was not tested.	Not verified

4.2 Accounts and keys

What	Needed?	Where
Gemini API key	Yes, for the answering step (not for retrieval)	Google AI Studio, https://aistudio.google.com (link from the post; it responded). The carousel says "No credit card needed".
Google account	Yes, for AI Studio and Colab	
Hugging Face account/token	No. The model is not gated.	
A test PDF you know well	Yes. You need to know which page answers which question.	You supply this

Free-tier rate limits: Google no longer publishes static RPM/RPD/TPM numbers. The rate-limits page only says limits "can be viewed in Google AI Studio" and links to a per-account dashboard behind login. Check your own numbers there before you plan a bulk run.

4.3 Pinned version set (verified 2026-09-28)

Package	Pin	Reason
Python	3.10 to 3.14 (3.12 recommended)	colpali-engine requires-python = ">=3.10, <3.15". The 3.12 choice is mine.
colpali-engine	<code>==0.3.18</code>	Latest; LoRA-conversion fix under transformers v5
transformers	<code>==5.17.0</code>	Latest; inside colpali-engine's <code>>=5.3.0, <6.0.0</code>
peft	<code>>=0.18.0, <0.21.0</code>	colpali-engine's own bound; currently resolves to 0.20.x

Package	Pin	Reason
torch	==2.13.0	Newest release under colpali-engine's <2.14.0 ceiling (released 2026-07-08)
torchvision	==0.28.0	Released with torch 2.13.0; cu126 wheels confirmed for Windows and Linux, Python 3.10 to 3.14
google-genai	==2.25.0	Latest (2026-09-22)
accelerate, pdf2image, Pillow, python-dotenv, numpy	unpinned	Versions not checked. colpali-engine requires pillow>=10.0.0. accelerate is needed for device_map.
poppler	Windows zip Release-26.09.0-0; Homebrew 26.09.0; apt poppler-utils (distro version)	

Nobody in this research ran this exact set end to end. The pins come from each package's declared bounds and release dates. If pip reports a conflict, the fallback is to let the resolver choose inside colpali-engine's bounds: `pip install colpali-engine==0.3.18 "torch<2.14"`.

requirements.txt:

```
colpali-engine==0.3.18
transformers==5.17.0
peft>=0.18.0,<0.21.0
torch==2.13.0
torchvision==0.28.0
accelerate
pdf2image
Pillow>=10.0.0
google-genai==2.25.0
python-dotenv
numpy
```

5. Environment setup

The GPU install is always two steps: first torch and torchvision from PyTorch's CUDA index, then everything else. torch==2.13.0 in requirements.txt is already satisfied by 2.13.0+cu126, so pip keeps the CUDA build. cu126 was checked for wheels. The same page also lists cu130, cu132 and cpu indexes for 2.13.0, but those were not checked file by file.

5.1 Google Colab

1. Runtime > Change runtime type > **T4 GPU**. Then check it:

```
!nvidia-smi
```

2. Install poppler and the Python packages:

```
!apt-get install -y poppler-utils -q
!pip install -q torch==2.13.0 torchvision==0.28.0 --index-url https://download.pytorch.org/whl/cu126
!pip install -q colpali-engine==0.3.18 transformers==5.17.0 "peft>=0.18.0,<0.21.0" accelerate pdf2image
"Pillow>=10.0.0" google-genai==2.25.0 python-dotenv numpy
```

3. If torch had already been imported in this session, or pip replaced Colab's preinstalled torch, use **Runtime > Restart session** before continuing.
4. Create the files. Put each file from section 7 in its own cell, starting with `%%writefile rag.py`, `%%writefile ingest.py` and so on. Then:

```
!mkdir -p pdfs index
```

Upload your PDF into pdfs/ with the Files panel on the left.

5. API key. Store it in Colab's Secrets panel (key icon) as GEMINI_API_KEY, then:

```
import os
from google.colab import userdata
os.environ["GEMINI_API_KEY"] = userdata.get("GEMINI_API_KEY")
```

6. Index, then query inside the notebook, so the model loads once and not on every question:

```
!python ingest.py pdfs/test.pdf
```

```
from query import make_client, ask
from rag import load_model, load_index

client = make_client()
model, processor = load_model()
embeddings, metadata = load_index("index")
print(ask("What was revenue growth between Q2 and Q3?", model, processor, embeddings, metadata, client))
```

Colab wipes the runtime disk when the session ends. The ~8.9 GB model downloads again each session, and index/ is lost. If that matters, mount Google Drive and copy index/ there. You can also point HF_HOME at a Drive folder, which uses ~9 GB of Drive quota.

5.2 Windows (PowerShell, NVIDIA GPU)

```
mkdir vision-rag; cd vision-rag
mkdir pdfs, index
py -3.12 -m venv .venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install torch==2.13.0 torchvision==0.28.0 --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

The last line should print 2.13.0+cu126 True. If it prints False, see troubleshooting. For a machine without an NVIDIA GPU, use `--index-url https://download.pytorch.org/whl/cpu`.

If you use uv, as the carousel did: `uv venv --python 3.12`, then `uv pip install` with the same arguments.

Poppler on Windows (there is no pip package or installer):

1. Open <https://github.com/oschwartz10612/poppler-windows/releases>
2. Download Release-26.09.0-0.zip (latest as of 2026-09-15), or whatever is newest.
3. Extract it and rename the folder to poppler. Put it in the project root, so the path is vision-rag\poppler\Library\bin\pdftoppm.exe.
4. Check it:

```
.\poppler\Library\bin\pdftoppm.exe -v
```

ingest.py finds this folder next to itself automatically.

Key: create .env in the project root:

```
GEMINI_API_KEY=your_key_here
```

5.3 Linux and macOS

```
# Linux (Debian/Ubuntu)
sudo apt-get install -y poppler-utils
# macOS
brew install poppler

mkdir -p vision-rag/pdfs vision-rag/index && cd vision-rag
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip

# Linux with an NVIDIA GPU:
pip install torch==2.13.0 torchvision==0.28.0 --index-url https://download.pytorch.org/whl/cu126
# macOS (CPU in this guide): skip the line above; requirements.txt pulls torch from PyPI

pip install -r requirements.txt
pdftoppm -v
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
echo "GEMINI_API_KEY=your_key_here" > .env
```

Poppler is on the system PATH here, so `_get_poppler_path()` returns None and pdf2image finds it.

5.4 Token cost of one question (estimate)

Google counts an image as 258 tokens per 768 x 768 tile. A letter page at 150 DPI is 1275 x 1650 pixels, which is about 6 tiles, or ~1,548 tokens. Three pages come to about 4,650 input tokens. At the paid rate that is about \$0.0035, plus about \$0.002 for a 500-token answer. This estimate comes from the research. It was not measured, and Gemini may resize images differently.

6. Project layout

```
vision-rag/
|-- .env                GEMINI_API_KEY=...
|-- requirements.txt
|-- rag.py              shared: model loader, index loader
|-- ingest.py           PDF -> page images -> embeddings -> index/
|-- query.py            question -> top-k pages -> Gemini -> cited answer (CLI)
|-- eval.py             retrieval metrics on your ground-truth set
|-- eval_set.json       your questions and correct pages
|-- baseline.py         whole-PDF comparison (section 2)
|-- pdfs/               your PDFs
|-- index/              emb_0.npy ... emb_N.npy + metadata.json
`-- poppler/            Windows only: poppler/Library/bin/pdftoppm.exe
```

One index folder holds one PDF. Running `ingest.py` again clears it. For several PDFs, use `--index-dir index/other` or see Extensions.

7. The code

The carousel's function names and signatures are kept: `load_model`, `load_index`, `_get_poppler_path`, `pdf_to_images`, `embed_pages`, `build_index`, `retrieve_pages`. The changes, and why:

- Embeddings are saved as float16. NumPy cannot hold bfloat16.
- Padding rows are trimmed with the attention mask before saving.
- Pages are rendered 16 at a time, so a long PDF does not fill RAM.
- The old index is cleared before a rebuild, so files and metadata stay in step.
- Query and page embeddings are both scored in float32.
- `metadata.json` also stores `pdf_path` and `dpi`, so `query.py` can re-render exactly the retrieved pages.

7.1 rag.py

```
# rag.py - shared model loader and index loader (used by ingest.py, query.py, eval.py)
import json
import os
import pathlib

import numpy as np
import torch
from colpali_engine.models import ColQwen2, ColQwen2Processor

MODEL_NAME = "vidore/colqwen2-v1.0"

def load_model() -> tuple[ColQwen2, ColQwen2Processor]:
    device = "cuda" if torch.cuda.is_available() else "cpu"
    dtype = torch.bfloat16 if device == "cuda" else torch.float32
    # Escape hatch for a GPU that rejects bfloat16: set VRAG_DTYPE=float32
    if os.environ.get("VRAG_DTYPE") == "float32":
        dtype = torch.float32

    model = ColQwen2.from_pretrained(
        MODEL_NAME,
        torch_dtype=dtype,
        device_map=device,
    ).eval() # eval() turns off dropout so embeddings are repeatable
    processor = ColQwen2Processor.from_pretrained(MODEL_NAME)

    print(f"Model loaded on {model.device} ({model.dtype})")
    return model, processor

def load_index(index_dir: str = "index") -> tuple[list[torch.Tensor], list[dict]]:
    index_path = pathlib.Path(index_dir)
    if not index_path.exists():
        raise FileNotFoundError(f"Index directory '{index_dir}' not found. Run ingest.py first.")

    # Numeric sort, so emb_10.npy comes after emb_9.npy and not after emb_1.npy
    npy_files = sorted(index_path.glob("emb_*.npy"), key=lambda p: int(p.stem.split("_")[1]))
```



```

if not npy_files:
    raise FileNotFoundError("No embeddings found. Run ingest.py first.")

# float16 on disk; scored in float32 so the dtype always matches the query
embeddings = [torch.from_numpy(np.load(f)).to(torch.float32) for f in npy_files]

with open(index_path / "metadata.json", encoding="utf-8") as f:
    metadata = json.load(f)

if len(metadata) != len(embeddings):
    raise ValueError(
        f"{len(embeddings)} embedding files but {len(metadata)} metadata entries. "
        "Re-run ingest.py to rebuild the index."
    )

print(f"Loaded index: {len(embeddings)} pages")
return embeddings, metadata

```

7.2 ingest.py

```

# ingest.py - PDF -> page images -> ColQwen2 page embeddings -> index/ on disk
# Usage: python ingest.py pdfs/report.pdf [--index-dir index] [--dpi 150] [--batch-size 2]
import argparse
import json
import pathlib
import time

import numpy as np
import torch
from colpali_engine.models import ColQwen2, ColQwen2Processor
from pdf2image import convert_from_path, pdfinfo_from_path
from PIL import Image

from rag import MODEL_NAME, load_model

def _get_poppler_path() -> str | None:
    """Return the path to poppler binaries on Windows, or None on Linux/macOS."""
    local = pathlib.Path(__file__).resolve().parent / "poppler" / "Library" / "bin"
    if local.exists():
        return str(local)
    return None

def pdf_to_images(
    pdf_path: str,
    dpi: int = 150,
    first_page: int | None = None,
    last_page: int | None = None,
) -> list[Image.Image]:
    # On Windows, point to the extracted poppler bin folder.
    # On Linux/macOS, poppler_path stays None (uses system PATH).
    pages = convert_from_path(
        pdf_path,
        dpi=dpi,
        first_page=first_page,

```

```

        last_page=last_page,
        poppler_path=_get_poppler_path(),
    )
    # The processor expects RGB; some PDFs render with an alpha channel or in greyscale
    return [page.convert("RGB") for page in pages]

def embed_pages(
    pages: list[Image.Image],
    model: ColQwen2,
    processor: ColQwen2Processor,
    batch_size: int = 2,
) -> list[torch.Tensor]:
    all_embeddings = []

    for i in range(0, len(pages), batch_size):
        batch = pages[i : i + batch_size]
        inputs = processor.process_images(batch).to(model.device)

        with torch.no_grad():
            embeddings = model(**inputs)

        # Each row is (sequence_length, 128). Rows past a shorter page's end are zero padding;
        # the attention mask drops them before the embedding leaves the GPU.
        for emb, mask in zip(embeddings, inputs["attention_mask"]):
            all_embeddings.append(emb[mask.bool()].cpu())

    return all_embeddings

def build_index(
    pdf_path: str,
    index_dir: str = "index",
    dpi: int = 150,
    batch_size: int = 2,
    chunk_pages: int = 16,
) -> None:
    pdf = pathlib.Path(pdf_path).resolve()
    if not pdf.exists():
        raise FileNotFoundError(f"PDF not found: {pdf}")

    index_path = pathlib.Path(index_dir)
    index_path.mkdir(parents=True, exist_ok=True)
    # One PDF per index folder: clear the previous run so files and metadata stay in step
    for old in index_path.glob("emb_*.npz"):
        old.unlink()
    (index_path / "metadata.json").unlink(missing_ok=True)

    n_pages = int(pdfinfo_from_path(str(pdf), poppler_path=_get_poppler_path())["Pages"])
    print(f"{pdf.name}: {n_pages} pages")

    print("Loading model...")
    model, processor = load_model()

    start = time.perf_counter()
    metadata = []

```

```
# Render a chunk of pages at a time so a long PDF never sits in RAM all at once
for first in range(1, n_pages + 1, chunk_pages):
    last = min(first + chunk_pages - 1, n_pages)
    pages = pdf_to_images(str(pdf), dpi=dpi, first_page=first, last_page=last)
    embeddings = embed_pages(pages, model, processor, batch_size=batch_size)

    for offset, emb in enumerate(embeddings):
        idx = first - 1 + offset
        # NumPy has no bfloat16; float16 halves disk use and is ample for unit-length vectors
        np.save(index_path / f"emb_{idx}.npy", emb.to(torch.float16).numpy())
        metadata.append({
            "index": idx,
            "pdf": pdf.name,
            "pdf_path": str(pdf),
            "page_number": idx + 1,
            "dpi": dpi,
            "n_vectors": int(emb.shape[0]),
            "model": MODEL_NAME,
        })
    print(f"Embedded pages {first} to {last} of {n_pages}")

with open(index_path / "metadata.json", "w", encoding="utf-8") as f:
    json.dump(metadata, f, indent=2)

elapsed = time.perf_counter() - start
print(
    f"Index saved: {len(metadata)} pages in '{index_dir}'/"
    f"({elapsed:.0f}s, {elapsed / max(len(metadata), 1):.1f}s per page)"
)
if torch.cuda.is_available():
    peak = torch.cuda.max_memory_allocated() / 1024**3
    print(f"Peak GPU memory allocated: {peak:.1f} GiB")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Index one PDF for visual retrieval.")
    parser.add_argument("pdf", help="path to the PDF, e.g. pdfs/report.pdf")
    parser.add_argument("--index-dir", default="index")
    parser.add_argument("--dpi", type=int, default=150)
    parser.add_argument("--batch-size", type=int, default=2)
    args = parser.parse_args()
    build_index(args.pdf, index_dir=args.index_dir, dpi=args.dpi, batch_size=args.batch_size)
```

7.3 query.py (the half the carousel withheld)

The Gemini call uses `client.models.generate_content`, which Google's API reference presents as current and supported. PIL images go straight into `contents`, as in Google's image-understanding guide. The client reads `GEMINI_API_KEY` from the environment. If `GOOGLE_API_KEY` is also set, it takes precedence. Each image is preceded by a text label with its page number, which is what lets Gemini cite pages correctly.

[illegible]

```

import os
import sys
import time

import torch
from colpali_engine.models import ColQwen2, ColQwen2Processor
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
from PIL import Image

from ingest import pdf_to_images
from rag import load_index, load_model

GEMINI_MODEL = "gemini-3.8-flash" # cheaper option: "gemini-3.5-flash-lite"

SYSTEM_PROMPT = """You answer questions about a document using only the page images provided.
Each image is preceded by a label with its file name and page number.

Rules:
- Use only what is visible on those pages. Do not use outside knowledge or guess.
- After each fact you state, cite the page it came from as (p. N), using the page number from the label.
- For numbers from a table or chart, name the row, column, series or axis the number belongs to.
- If the pages do not contain the answer, reply: "Not found in the retrieved pages (checked p. X, p. Y).".
- Keep the answer short. Quote figures exactly as printed."""

def retrieve_pages(
    query: str,
    model: ColQwen2,
    processor: ColQwen2Processor,
    embeddings: list[torch.Tensor],
    metadata: list[dict],
    top_k: int = 3,
) -> list[dict]:
    inputs = processor.process_queries([query]).to(model.device)
    with torch.no_grad():
        query_emb = model(**inputs)

    # Page embeddings are float32 on CPU (see load_index); match the query, then let
    # score_multi_vector move both to the model's device for the MaxSim sum
    query_emb = query_emb.to(torch.float32)
    scores = processor.score_multi_vector(query_emb, embeddings, device=model.device)
    scores = scores[0].tolist()

    ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True)

    results = []
    for idx, score in ranked[:top_k]:
        results.append(**metadata[idx], "score": score)
    return results

def load_page_images(results: list[dict]) -> list[Image.Image]:
    """Re-render only the retrieved pages from the original PDF, at the DPI used for indexing."""
    images = []

```

```

for r in results:
    n = r["page_number"]
    images.extend(pdf_to_images(r["pdf_path"], dpi=r["dpi"], first_page=n, last_page=n))
return images

def answer_with_gemini(
    client: genai.Client,
    question: str,
    results: list[dict],
    images: list[Image.Image],
    model_id: str = GEMINI_MODEL,
    max_attempts: int = 4,
) -> str:
    contents: list = []
    for r, img in zip(results, images):
        contents.append(f"Page {r['page_number']} of {r['pdf']}:")
        contents.append(img)
    contents.append(f"Question: {question}")

    config = types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)

    for attempt in range(1, max_attempts + 1):
        try:
            response = client.models.generate_content(
                model=model_id,
                contents=contents,
                config=config,
            )
            return response.text or "(Gemini returned no text; inspect response.candidates for the reason.)"
        except errors.APIError as e:
            # 429 = rate limited, 500/503 = transient server trouble; anything else is a real error
            if e.code in (429, 500, 503) and attempt < max_attempts:
                wait = 5 * 2**attempt
                print(f"Gemini returned {e.code}; retrying in {wait}s ({attempt}/{max_attempts - 1})")
                time.sleep(wait)
                continue
            raise
    raise RuntimeError("unreachable")

def print_results(results: list[dict]) -> None:
    print("Retrieved pages:")
    for rank, r in enumerate(results, start=1):
        print(f"  {rank}. {r['pdf']} p.{r['page_number']}  score={r['score']:.2f}")

def make_client() -> genai.Client:
    load_dotenv()
    if not (os.environ.get("GEMINI_API_KEY") or os.environ.get("GOOGLE_API_KEY")):
        sys.exit("No Gemini API key. Put GEMINI_API_KEY=... in .env, or run with --retrieve-only.")
    return genai.Client() # reads GEMINI_API_KEY; GOOGLE_API_KEY wins if both are set

def ask(

```

```

question: str,
model: ColQwen2,
processor: ColQwen2Processor,
embeddings: list[torch.Tensor],
metadata: list[dict],
client: genai.Client | None,
top_k: int = 3,
model_id: str = GEMINI_MODEL,
) -> str:
    results = retrieve_pages(question, model, processor, embeddings, metadata, top_k=top_k)
    print_results(results)
    if client is None:
        return ""
    images = load_page_images(results)
    return answer_with_gemini(client, question, results, images, model_id=model_id)

def main() -> None:
    parser = argparse.ArgumentParser(description="Ask a question about the indexed PDF.")
    parser.add_argument("question", nargs="?", help="leave out for an interactive prompt")
    parser.add_argument("--index-dir", default="index")
    parser.add_argument("--top-k", type=int, default=3)
    parser.add_argument("--gemini-model", default=GEMINI_MODEL)
    parser.add_argument("--retrieve-only", action="store_true", help="show the pages, skip Gemini")
    args = parser.parse_args()

    # Check the key before spending a minute loading the model
    client = None if args.retrieve_only else make_client()
    model, processor = load_model()
    embeddings, metadata = load_index(args.index_dir)

    def run(q: str) -> None:
        answer = ask(q, model, processor, embeddings, metadata, client,
                     top_k=args.top_k, model_id=args.gemini_model)
        if answer:
            print("\nAnswer:\n" + answer)

    if args.question:
        run(args.question)
        return

    print("Type a question (empty line to quit).")
    while True:
        try:
            q = input("\n> ").strip()
        except EOFError:
            break
        if not q:
            break
        run(q)

if __name__ == "__main__":
    main()

```

7.4 End-to-end run

```
python ingest.py pdfs/test.pdf
python query.py "What was revenue growth between Q2 and Q3?" --retrieve-only
python query.py "What was revenue growth between Q2 and Q3?"
```

The output should have this shape. The numbers are placeholders.

```
test.pdf: 42 pages
Loading model...
Model loaded on cuda:0 (torch.bfloat16)
Embedded pages 1 to 16 of 42
...
Index saved: 42 pages in 'index/' (...s, ...s per page)
Peak GPU memory allocated: ... GiB

Model loaded on cuda:0 (torch.bfloat16)
Loaded index: 42 pages
Retrieved pages:
  1. test.pdf p.14  score=...
  2. test.pdf p.15  score=...
  3. test.pdf p.3   score=...

Answer:
Revenue rose from 43.2 in Q2 to 51.8 in Q3 (Revenue row, Q2 and Q3 columns) (p. 14).
```

8. Build order and milestone checks

Times are rough and hardware-dependent. Embedding speed is not published for this checkpoint, so `ingest.py` prints your real seconds per page and peak VRAM. Start with a 10 to 20 page PDF.

#	Milestone	Command	It worked if	Rough time	Cost
0	Environment	<code>python -c "import torch, transformers, colpali_engine; print(torch.__version__, transformers.__version__, torch.cuda.is_available())"</code> and <code>pdftoppm -v</code>	2.13.0+cu126 5.17.0 True (CPU installs: False), and poppler prints its version	20 to 40 min	\$0
1	Model loads	<code>python -c "from rag import load_model; load_model()"</code>	Model loaded on cuda:0 (torch.bfloat16) (CPU: cpu (torch.float32))	Download of ~8.9 GB: ~12 min at 100 Mbit/s, ~2.5 min at 500 Mbit/s. Later loads come from the cache.	\$0

#	Milestone	Command	It worked if	Rough time	Cost
2	N pages embedded	<code>python ingest.py pdfs/test.pdf</code>	test.pdf: N pages, one Embedded pages ... line per 16 pages, then Index saved: N pages with the same N	Not published; read the s/page line	\$0 on Colab free tier or your own GPU
3	Index on disk	<code>python -c "import numpy as np, json; m=json.load(open('index/metadata.json')); a=np.load('index/emb_0.npy'); print(len(m), a.shape, a.dtype)"</code>	N (≤ 768 , 128) float16. The directory has emb_0.npy to emb_{N-1}.npy, each ≤ 192 KiB.	Seconds	\$0
4	Retrieval finds the known page	Pick a question whose answer you know is on page P: <code>python query.py "..."</code> --retrieve-only	Page P is rank 1, or at least in the top 3. Try 5 questions, including a table and a chart one.	5 to 15 min	\$0 (no API call)
5	Cited end-to-end answer	<code>python query.py "..."</code>	The answer is correct and cites (p. P). A question the PDF cannot answer returns "Not found in the retrieved pages".	10 min	Free tier \$0; paid about \$0.005 per question (estimate)
6	Measured quality	<code>python eval.py eval_set.json</code> (section 10)	A Recall@3 and MRR you can compare against later changes and against baseline.py	1 to 2 h to write questions	\$0

9. Troubleshooting

Symptom	Cause	Fix
PDFInfoNotInstalledError / "Unable to get page count. Is poppler installed and in PATH?"	pdf2image cannot find poppler	Colab/Linux: <code>apt-get install -y poppler-utils</code> . macOS: <code>brew install poppler</code> . Windows: check that vision-rag\poppler\Library\bin\pdftoppm.exe exists (the folder was renamed to poppler and the zip's inner layout is intact), and run <code>.\poppler\Library\bin\pdftoppm.exe -v</code> .
PDFPageCountError / PDFSyntaxError	Damaged or encrypted PDF	Open it in a viewer; re-save or decrypt it first

Symptom	Cause	Fix
<code>torch.OutOfMemoryError: CUDA out of memory during ingest</code>	Batch too large for the card	Cut batch size first: <code>--batch-size 1</code> . The embeddings do not change. Lowering DPI barely helps VRAM: the processor already scales every page down to its 602,112-pixel cap (768 patches), which is about 80 DPI for a letter or A4 page. DPI above that affects CPU RAM and the image Gemini reads, not the model. If batch size 1 still fails, use a larger GPU or Colab's T4. Do not switch to <code>VRAG_DTYPE=float32</code> , which roughly doubles weight memory.
OOM in a notebook after it worked once	Two models loaded (for example an in-notebook <code>load_model()</code> and then <code>!python ingest.py</code>)	Runtime > Restart session; load one model at a time
<code>TypeError: Got unsupported ScalarType BFloat16 on save</code>	The carousel's <code>emb.numpy()</code> on a <code>bfloat16</code> tensor	Use the <code>ingest.py</code> above, which casts to <code>float16</code> first
<code>RuntimeError: expected ... same dtype / einsum dtype errors in score_multi_vector</code>	Query in <code>bfloat16</code> , pages in <code>float16</code> or <code>float32</code>	Use <code>load_index</code> and <code>retrieve_pages</code> above; both cast to <code>float32</code>
Expected all tensors to be on the same device	Inputs left on CPU while the model is on GPU	Keep <code>.to(model.device)</code> on processor outputs and <code>device=model.device</code> in <code>score_multi_vector</code>
<code>bfloat16</code> errors or garbage output on an older GPU	Card or kernel lacks <code>bfloat16</code> support (not verified for any specific card)	<code>VRAG_DTYPE=float32 python ingest.py ...</code> Needs about 2x the VRAM for weights. Re-ingest after switching.
<code>torch.cuda.is_available()</code> is <code>False</code> on a machine with an NVIDIA GPU	CPU-only wheel installed (the PyPI default on Windows), or the driver is too old	Reinstall torch with <code>--index-url https://download.pytorch.org/whl/cu126</code> . <code>nvidia-smi</code> must show a CUDA version of 12.6 or higher.
First run "hangs" for minutes	The ~8.9 GB download (not ~5 GB as the carousel says) into the Hugging Face cache (<code>~/.cache/huggingface</code> ; on Windows <code>%USERPROFILE%\cache\huggingface</code>)	Wait. Re-running resumes the download. Set <code>HF_HOME</code> to a drive with 10+ GB free before the first run if your system drive is small.
Disk full during download	Cache on a small drive	Set <code>HF_HOME=D:\hf-cache</code> (or similar), delete the partial download, re-run
No Gemini API key... from <code>query.py</code>	<code>.env</code> missing, or not in the folder you run from	Create <code>.env</code> with <code>GEMINI_API_KEY=...</code> in the working directory, or set the variable. In Colab, use the Secrets snippet.
<code>APIError 400 or 403, "API key not valid"</code>	Wrong or revoked key, or a stale <code>GOOGLE_API_KEY</code> overriding it	Make a new key in AI Studio. Unset <code>GOOGLE_API_KEY</code> , which takes precedence over <code>GEMINI_API_KEY</code> .

Symptom	Cause	Fix
APIError 429 / RESOURCE_EXHAUSTED	Free-tier rate limit	query.py retries after 10, 20 and 40 s. For bulk runs, check your limits at AI Studio's rate-limit dashboard, slow down, try --gemini-model gemini-3.5-flash-lite, or enable billing.
APIError 404 on the model	Model ID typo, or gemini-2.5-flash (the carousel's model), which is now limited to accounts that used it before	Use gemini-3.8-flash or gemini-3.5-flash-lite
ImportError: cannot import name '_maybe_shard_state_dict_for_tp'	transformers newer than the installed peft expects. Upstream, this was a dev-build mismatch (transformers #45405). It is not a reason to pin 5.3.0.	pip show transformers peft, then pip install "peft>=0.18.0,<0.21.0" transformers==5.17.0
pip ResolutionImpossible mentioning torch	Something pulled torch 2.14.0; colpali-engine 0.3.18 requires <2.14.0	Install torch==2.13.0 torchvision==0.28.0 first from the cu126 index, then the rest
pip conflict on transformers	Forcing transformers outside [5.3.0, 6.0.0), e.g. 4.x	Use 5.17.0, or omit the pin and let colpali-engine choose
Retrieval ranks look random, no error	Possibly an older colpali-engine (0.3.17 or older) under transformers v5 silently dropping the LoRA weights (fixed in 0.3.18)	pip show colpali-engine must show 0.3.18. Re-ingest after upgrading.
ValueError: N embedding files but M metadata entries	An interrupted or partial ingest	Re-run ingest.py; it clears the folder first
.\.venv\Scripts\Activate.ps1 cannot be loaded (Windows)	PowerShell execution policy	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, then re-activate
Right page retrieved, answer still wrong	Small print unreadable at 150 DPI, or the answer spans pages	Re-ingest with --dpi 200. This changes only the image Gemini reads, not retrieval (see the OOM row). Or use --top-k 5.

10. Evaluation: is it actually working?

Build a small ground-truth set (30 to 50 questions)

1. Choose 3 to 5 PDFs you know. Cover a table-heavy page, a chart page, a multi-column layout, a scanned page if you have one, and some plain text.
2. Write 6 to 12 questions per PDF, as you would really ask them. Include questions whose answer needs the table or chart structure. Those are the reason for choosing this approach.
3. Read the PDF and write down the correct page number(s). Allow more than one page where a table continues onto the next page.
4. Keep the mix balanced. If every question comes from one document type, the number will not tell you about the others.

eval_set.json:

```
[
  {"question": "What was revenue growth between Q2 and Q3?", "pdf": "test.pdf", "correct_pages": [14],
  "type": "table"},
  {"question": "Which region had the highest share in the 2025 pie chart?", "pdf": "test.pdf",
  "correct_pages": [22], "type": "chart"},
  {"question": "What is the warranty period for replacement parts?", "pdf": "test.pdf", "correct_pages":
  [37, 38], "type": "text"}
]
```

pdf must match the file name stored in metadata.json.

Metrics to report

- **Recall@3** is the headline, because query.py sends the top 3 pages. It is the share of correct pages that reach Gemini. If this is low, no prompt will save the answer.
- **MRR@10** (1 / rank of the first correct page, averaged) is a single number for "how near the top".
- **nDCG@5** matters when questions have several correct pages.
- **Answer accuracy**, graded by hand: for each question, is the answer right, and is the cited page right? A wrong answer with the right page retrieved is a generation problem (DPI, prompt). A wrong page retrieved is a retrieval problem.

Run the same questions through baseline.py and compare answer accuracy. That comparison tells you whether the index earns its keep on your documents. Published scores (ViDoRe, and the vendor figures in section 11) come from other corpora and will not transfer exactly.

eval.py

```
# eval.py - retrieval quality on a hand-built question set
# Usage: python eval.py eval_set.json [--index-dir index]
import argparse
import json
import math

from query import retrieve_pages
from rag import load_index, load_model

K_VALUES = (1, 3, 5)
DEPTH = 10

def evaluate(eval_path: str, index_dir: str = "index") -> None:
    with open(eval_path, encoding="utf-8") as f:
        items = json.load(f)

    model, processor = load_model()
    embeddings, metadata = load_index(index_dir)

    totals = {f"recall@{k}": 0.0 for k in K_VALUES}
    totals[f"mrr@{DEPTH}"] = 0.0
    totals["ndcg@5"] = 0.0
    misses = []

    for item in items:
        correct = {(item["pdf"], p) for p in item["correct_pages"]}
        results = retrieve_pages(item["question"], model, processor, embeddings, metadata, top_k=DEPTH)
        ranked = [(r["pdf"], r["page_number"]) for r in results]
```

```

for k in K_VALUES:
    totals[f"recall@{k}"] += len(correct & set(ranked[:k])) / len(correct)

first_hit = next((i for i, key in enumerate(ranked, start=1) if key in correct), None)
totals[f"mrr@{DEPTH}"] += 1 / first_hit if first_hit else 0.0

dcg = sum(1 / math.log2(i + 1) for i, key in enumerate(ranked[:5], start=1) if key in correct)
idcg = sum(1 / math.log2(i + 1) for i in range(1, min(len(correct), 5) + 1))
totals["ndcg@5"] += dcg / idcg

if first_hit is None or first_hit > 3:
    misses.append((item["question"], sorted(item["correct_pages"]), [p for _, p in ranked[:3]]))

n = len(items)
print(f"\n{n} questions")
for name, value in totals.items():
    print(f"  {name:10s} {value / n:.3f}")

if misses:
    print("\nCorrect page not in the top 3:")
    for question, want, got in misses:
        print(f"  - {question}\n      correct {want}, got {got}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Score retrieval against eval_set.json")
    parser.add_argument("eval_set")
    parser.add_argument("--index-dir", default="index")
    args = parser.parse_args()
    evaluate(args.eval_set, index_dir=args.index_dir)

```

The eval set above indexes one PDF per folder. For several PDFs in one eval, build a combined index first (Extensions).

Formal option: vidore-benchmark (illuin-tech) evaluates full pipelines with pytrec_eval metrics. You subclass BasePipeline and implement index() and search(). Its README only documents Hub-hosted ViDoRe datasets, so using your own PDFs means packaging them in the same corpus/queries/qrels shape. For 30 to 50 questions, eval.py is simpler.

11. Scaling up

Memory for page embeddings (768 patches x 128 dims, the practical cap at 150 DPI)

Pages	float32 (in RAM as load_index holds them)	float16 (on disk as ingest.py writes them)
100	37.5 MiB	18.75 MiB
1,000	375 MiB	187.5 MiB
10,000	3.66 GiB	1.83 GiB

This is per page: 768 x 128 x 4 bytes = 384 KiB in float32, and 192 KiB in float16. The model card's example page used 755 patches, less than 2% under the cap. These figures are for embeddings only; the model weights come on top.

What breaks first in this design:

- **Up to a few thousand pages:** fine as written. score_multi_vector scores pages in batches of 128 on the GPU.

- **~10,000 pages:** 3.7 GiB of float32 in RAM, 10,000 small files, and a full MaxSim scan on every query. That is the point to move to a store.

Which store

Scale	Store	Why
Hundreds of pages, one machine	Stay on files, or LanceDB (embedded, no server)	Native multivector MaxSim (cosine only), nested-list column <code>pa.list_(pa.list_(pa.float32(), 128))</code> . No written first-party ColPali guide.
~10,000 pages, a server	Qdrant (easiest to stand up) or Vespa (most documented at large scale)	Both natively support MaxSim and have first-party ColPali/ColQwen guides with published numbers
Milvus	Works, but not natively	Patches are stored as rows and MaxSim is computed client-side (<code>MilvusColbertRetriever</code>)
Weaviate (v1.29+)	Native	<code>Configure.MultiVectors.self_provided(...)</code> ; has a ColQwen2 recipe

Qdrant config, adapted from Qdrant's "Multivectors and Late Interaction" page to this index. I have not run it. Install with `pip install qdrant-client` (version not checked) and run a Qdrant server as its docs describe.

```
import torch
from qdrant_client import QdrantClient, models

from rag import load_index

client = QdrantClient(url="http://localhost:6333")
client.create_collection(
    collection_name="pdf_pages",
    vectors_config={
        "original": models.VectorParams(
            size=128,
            distance=models.Distance.COSINE,
            multivector_config=models.MultiVectorConfig(
                comparator=models.MultiVectorComparator.MAX_SIM
            ),
            hnsw_config=models.HnswConfigDiff(m=0), # no graph on raw patches; used for rerank
        ),
    },
)

embeddings, metadata = load_index("index")
points = [
    models.PointStruct(id=m["index"], vector={"original": emb.tolist()}, payload=m)
    for emb, m in zip(embeddings, metadata)
]
client.upload_points(collection_name="pdf_pages", points=points, batch_size=8)

# query_emb: model(**processor.process_queries([q]).to(model.device)) as in retrieve_pages
hits = client.query_points(
    collection_name="pdf_pages",
    query=query_emb[0].to("cpu", dtype=torch.float32).tolist(),
    using="original",
    limit=3,
```

```

    with_payload=True,
)

```

With `m=0` and no prefetch, this is still a full MaxSim scan. Past a few thousand pages, add the two-stage design below.

Vespa stores binarised patches as `field colbert type tensor<int8>(page{ }, patch{ }, v[16])` (128 bits per patch). Ranking has two phases: a cheap first phase (BM25, or Hamming MaxSim), then float MaxSim over the survivors with `rerank_count=10`.

Compression and two-stage retrieval (published numbers)

Option	Source	Effect
Hierarchical token pooling, factor 3 (<code>HierarchicalTokenPooler</code> in <code>colpali-engine</code>)	colpali-engine	66.7% fewer vectors, 97.8% of the original performance
Row/column mean pooling for a first-stage HNSW prefetch (top 100 to 200), then full MaxSim rerank	Qdrant blog	~1,030 down to 38 vectors per page for stage 1. NDCG@20 0.952, Recall@20 0.917 vs full search. 13x faster.
Binary quantisation (128 floats to 128 bits)	Vespa	32x smaller than float32. nDCG@5 52.4 (float) to 49.5 (binary only), recovering to 51.6 with binary retrieval plus float rerank.
Binary quantisation on Qdrant multivectors	Qdrant	"Nearly half" the search time of scalar quantisation at "similar accuracy". No exact accuracy loss published.
int8 quantisation	none found	No vendor publishes ColPali/ColQwen2 int8 quality numbers
float16 storage	this guide	Already done in <code>ingest.py</code> ; halves disk

Re-run `eval.py` before and after any of these. The published deltas come from other corpora.

12. Extensions worth doing next

- Multiple PDFs with real metadata.** Change `build_index` to append instead of clearing: keep a running `idx` across PDFs, and for each PDF record `doc_id` (for example the first 16 hex characters of the file's SHA-256), `title` (pdfinfo's `Title` when present), `pages`, `pdf_path` and `ingested_at`. Skip files whose hash is already in `metadata.json`. In `retrieve_pages`, filter embeddings/metadata by `pdf` before scoring when the user names a document. `eval.py` already matches on (`pdf`, `page_number`).
- Cache page images.** Save each rendered page during ingest, and read it back at query time instead of re-running poppler:

```

# in build_index, inside the chunk loop (make the folder once before the loop):
# (index_path / "pages").mkdir(exist_ok=True)
for offset, page in enumerate(pages):
    page.save(index_path / "pages" / f"page_{first - 1 + offset}.jpg", quality=90)

```

```

# in query.py, a drop-in alternative to load_page_images
def load_cached_images(results: list[dict], index_dir: str = "index") -> list[Image.Image]:
    return [Image.open(f"{index_dir}/pages/page_{r['index']}.jpg").convert("RGB") for r in results]

```

This costs disk (roughly the JPEG size of each page) and saves a poppler call per retrieved page.

3. **Hybrid text + visual retrieval.** Poppler is already installed: `pdftotext -f N -l N -layout file.pdf` - gives each page's text. Index that with BM25, then merge the BM25 ranking and the MaxSim ranking with reciprocal rank fusion (score = sum of $1 / (60 + \text{rank})$, where 60 is a common constant). This is the same idea as Vespa's BM25 first phase. It helps with exact identifiers, part numbers and names that a vision model can blur.
4. **Swap in a newer checkpoint.** colpali-engine is deprecated. The maintained path is sentence-transformers[image]>=6.0.0, using the pattern from the colpali README:

```
from sentence_transformers import MultiVectorEncoder

model = MultiVectorEncoder("vidore/colqwen2-v1.0")
query_embeddings = model.encode_query(["Which year has the highest total outlay?"])
document_embeddings = model.encode_document(["page_1.png", "page_2.png"])
scores = model.similarity(query_embeddings, document_embeddings)
```

Candidates, with legacy ViDoRe averages from colpali-engine's README:

Checkpoint	Licence	Legacy ViDoRe avg	Note
vidore/colqwen2-v1.0 (this guide)	Apache 2.0	89.3	~2B
TomoroAI/tomoro-colqwen3-embed-4b	Apache 2.0	90.6	Research pick for a 16 GB T4. VRAM not stated on the card; 4B in bf16 is ~8 GB of weights by arithmetic.
athrael-soju/colqwen3.5-4.5B-v3	Apache 2.0	90.9	8.66 GB on disk; one individual's account, so treat maintenance as less certain
vidore/colqwen2.5-v0.2	Qwen Research Licence (non-commercial backbone)	89.4	Check the licence before any commercial use
nvidia/nemotron-colembed-vl-8b-v2	CC-BY-NC-4.0	n/a (No. 1 on ViDoRe V3, 63.54 nDCG@10, self-reported)	Non-commercial

Switching models means re-ingesting. Embeddings from different models cannot be compared. This guide did not run the Sentence Transformers path, and `score_multi_vector` does not apply there.

5. **A small UI.** Wrap `ask()` in a web form with one text box, an answer panel, and a gallery of the retrieved page images, so you can see what Gemini saw. Load the model and index once at startup, as `main()` does. Gradio and Streamlit both fit, but their current APIs were not checked for this guide, so no code is given.

13. Sources

Pages fetched during the research for this guide:

Model, library, versions

- <https://arxiv.org/abs/2407.01449>
- <https://pypi.org/pypi/colpali-engine/json>
- <https://github.com/illuin-tech/colpali>
- <https://github.com/illuin-tech/colpali/releases>

- <https://github.com/illuin-tech/colpali/blob/main/CHANGELOG.md>
- <https://raw.githubusercontent.com/illuin-tech/colpali/main/README.md>
- <https://raw.githubusercontent.com/illuin-tech/colpali/v0.3.18/pyproject.toml>
- https://raw.githubusercontent.com/illuin-tech/colpali/v0.3.18/colpali_engine/models/init.py
- https://raw.githubusercontent.com/illuin-tech/colpali/v0.3.18/colpali_engine/utils/processing_utils.py
- <https://pypi.org/project/transformers/#history>
- <https://github.com/huggingface/transformers/pull/48485>
- <https://github.com/huggingface/transformers/issues/45405>
- <https://github.com/huggingface/peft/issues/3628>
- https://github.com/search?q=%22_maybe_shard_state_dict_for_tp%22&type=issues
- <https://pypi.org/pypi/torch/json>
- <https://pypi.org/project/torch/#history>
- <https://pypi.org/pypi/torchvision/json>
- <https://pypi.org/project/torchvision/#history>
- <https://github.com/pytorch/pytorch/releases>
- <https://github.com/pytorch/vision/releases>
- <https://pytorch.org/blog/pytorch-2-14-release-blog/>
- <https://pytorch.org/get-started/locally/>
- <https://pytorch.org/get-started/previous-versions/>
- <https://download.pytorch.org/whl/cu126/torch/>
- <https://download.pytorch.org/whl/cu126/torchvision/>
- <https://huggingface.co/vidore>
- <https://huggingface.co/vidore/colqwen2-v1.0>
- <https://huggingface.co/vidore/colqwen2-v1.0/raw/main/README.md>
- <https://huggingface.co/api/models/vidore/colqwen2-v1.0?blobs=true>
- https://huggingface.co/vidore/colqwen2-v1.0/raw/main/adapters_config.json
- https://huggingface.co/vidore/colqwen2-v1.0/resolve/main/preprocessor_config.json
- <https://huggingface.co/api/models/vidore/colqwen2-base?blobs=true>
- <https://huggingface.co/vidore/colqwen2-base/raw/main/config.json>
- <https://huggingface.co/api/models/Qwen/Qwen2-VL-2B-Instruct?blobs=true>
- <https://github.com/Belval/pdf2image>

Poppler

- <https://github.com/oschwartz10612/poppler-windows/releases>
- <https://api.github.com/repos/oschwartz10612/poppler-windows/releases/latest>
- <https://github.com/oschwartz10612/poppler-windows/releases.atom>
- <https://formulae.brew.sh/formula/poppler>

Gemini

- <https://pypi.org/project/google-genai/>
- <https://pypi.org/project/google-generativeai/>
- <https://googleapis.github.io/python-genai/>
- <https://ai.google.dev/gemini-api/docs/libraries>
- <https://ai.google.dev/gemini-api/docs/migrate>
- <https://ai.google.dev/gemini-api/docs/deprecations>
- <https://ai.google.dev/gemini-api/docs/models>

- <https://ai.google.dev/gemini-api/docs/models/gemini-3.8-flash>
- <https://ai.google.dev/gemini-api/docs/pricing>
- <https://ai.google.dev/gemini-api/docs/rate-limits>
- <https://aistudio.google.com/rate-limit> (redirects to a Google login; not readable without an account)
- <https://aistudio.google.com> (from the post)
- <https://ai.google.dev/gemini-api/docs/text-generation>
- <https://ai.google.dev/api/generate-content>
- <https://ai.google.dev/api/interactions-api>
- <https://ai.google.dev/gemini-api/docs/generate-content/image-understanding>
- <https://ai.google.dev/gemini-api/docs/document-processing>
- <https://ai.google.dev/gemini-api/docs/caching>
- <https://ai.google.dev/gemini-api/docs/generate-content/caching>

Scaling and evaluation

- <https://qdrant.tech/documentation/tutorials-search-engineering/using-multivector-representations/>
- <https://qdrant.tech/documentation/tutorials-search-engineering/pdf-retrieval-at-scale/>
- <https://qdrant.tech/blog/colpali-qdrant-optimization/>
- <https://blog.vespa.ai/retrieval-with-vision-language-models-colpali/>
- <https://blog.vespa.ai/scaling-colpali-to-billions/>
- <https://vespa.ai/solutions/visual-retrieval-augmented-generation/colpali/>
- https://milvus.io/docs/use_ColPali_with_milvus.md
- <https://docs.lancedb.com/search/multivector-search>
- <https://docs.weaviate.io/weaviate/tutorials/multi-vector-embeddings>
- <https://docs.weaviate.io/weaviate/recipes/multi-vector-colipali-rag>
- <https://github.com/illuin-tech/vidore-benchmark>

Checkpoints and parsers

- <https://huggingface.co/vidore/colqwen2.5-v0.2>
- <https://huggingface.co/vidore/colpali-v1.2>
- <https://huggingface.co/vidore/colpali-v1.3>
- <https://huggingface.co/google/paligemma-3b-mix-448>
- <https://huggingface.co/nomic-ai/colnomic-embed-multimodal-3b>
- <https://huggingface.co/nomic-ai/colnomic-embed-multimodal-7b>
- <https://huggingface.co/jinaai/jina-embeddings-v4>
- <https://jina.ai/news/jina-embeddings-v4-universal-embeddings-for-multimodal-multilingual-retrieval>
- <https://huggingface.co/MrLight/dse-qwen2-2b-mrl-v1>
- <https://huggingface.co/nvidia/nemotron-colembed-vl-8b-v2>
- <https://huggingface.co/nvidia/llama-nemotron-colembed-vl-3b-v2>
- <https://huggingface.co/collections/nvidia/nemotron-colembed-v2>
- <https://huggingface.co/TomoroAI/tomoro-colqwen3-embed-4b>
- <https://huggingface.co/athrael-soju/colqwen3.5-4.5B-v3>
- <https://github.com/docling-project/docling>
- <https://docling-project.github.io/docling/>
- <https://huggingface.co/ibm-granite/granite-docling-258M>
- <https://github.com/datalab-to/marker>
- <https://github.com/Unstructured-IO/unstructured>

- <https://unstructured.io/>
- <https://unstructured.io/pricing>
- <https://mistral.ai/news/mistral-ocr>
- <https://docs.mistral.ai/models/ocr-4-1>
- <https://docs.mistral.ai/resources/changelogs>
- <https://mistral.ai/pricing/api/>

The carousel's "comment RAG" guide was not pursued. No official Learnbay repository was found, and none of the similarly named community repositories found in search are linked here.

14. What stayed unverified

- **Nobody ran this code or this exact version set.** The pins come from declared package bounds and release dates. The code follows the libraries' documented APIs, but it has not been executed. Expect to fix at most small API details on the first run.
- **Minimum VRAM** for `vidore/colqwen2-v1.0`: no primary source states one. The ~4.4 GB bf16 weight figure is arithmetic. Embedding speed per page is not published either.
- **Free-tier Gemini rate limits** (RPM, RPD, TPM): only visible on a logged-in AI Studio dashboard.
- **Whether a brand-new key can use gemini-2.5-flash**: the docs say access is limited to past users but give no cutoff.
- **Image token cost per retrieved page** (~1,548 tokens at 150 DPI): an estimate from Google's tiling rule, not measured. Gemini or the SDK may resize images.
- **CUDA indexes other than cu126** for torch 2.13.0 (cu130, cu132, cpu): listed on PyTorch's previous-versions page, whose render was partly garbled (it showed a wrong torchvision number). cu126 wheels were confirmed file by file. The minimum NVIDIA driver for cu126 was not checked.
- `apt-get install poppler-utils` was not checked against a Debian/Ubuntu package page. It is the carousel's own working command and matches pdf2image's README.
- **PyTorch refusing .numpy() on bfloat16** comes from PyTorch's known behaviour, not a page fetched for this guide.
- **transformers v5 and the torch_dtype argument**: `colpali-engine`'s current README still uses `torch_dtype=`, so this guide does too. If transformers warns that it is deprecated, rename it to `dtype=`.
- **Behaviour on T4 with bfloat16**, and on Apple MPS: not tested. `VRAG_DTYPE=f1oat32` is the fallback.
- **Sentence Transformers MultiVectorEncoder** beyond the README snippet, **Qdrant code** (adapted, not run), **Gradio/Streamlit** APIs, and the **qdrant-client version**: not checked.
- **Vendor quality numbers** (Qdrant, Vespa, `colpali-engine` pooling, Docling, marker) are self-reported by each vendor on their own benchmarks.
- **Licence of nomic-ai/colnomic-embed-multimodal-3b**: not stated on its card.